

Pushing the Pi5 Cluster

An empirical investigation of the bit-exact ceiling

Findings from 4 background Linux research agents and ~ 20 cluster experiments

Companion to “Distributed LLM Inference on a 4-Node Raspberry Pi 5 Cluster”

Daniel Correa Villa

Hellomatik — Raspberry Pi Cluster Lab

daniel.correa@hellomatik.com

17 May 2026

Abstract

This companion paper documents a multi-round investigation of the Raspberry Pi 5 cluster (4×Pi 5 16 GB, Cortex-A76, Gigabit Ethernet) running `distributed-llama` on Qwen3-30B-A3B Q40. Building on the original report’s 12.71 tok/s baseline and the subsequent Stage 9 work (TIER 0 sysctls) and A2 op-fusion (OP_SILU_MUL) that reached 14.011 tok/s, we conducted Rounds 3 and 4 of experiments: four parallel background research agents examined Linux Plumbers, SOSP, OSDI, EuroSys, eBPF, scheduler, and ARM-specific papers (2024–2026); we cherry-picked the Phase B async-sync foundation; implemented EEVDF per-task slice tuning via `sched_setattr`; applied multiple sysctl bundles; and tested ~ 15 distinct configurations including threaded NAPI, IRQ pinning, per-thread CPU affinity, and `-tile-sync` K-values. **Net measured improvement Rounds 3+4: 0% within noise.** The final cluster sustains 14.046 ± 0.049 tok/s (n=20, cold-start), a cumulative improvement of **+2.38% over the original 13.720 baseline** and **+7.72% above the publicly documented ceiling of 13.04 tok/s**. The 49% backend stall rate (measured via ARM PMU `stalled-cycles-backend`) confirms the cluster is DRAM-bandwidth-bound. Further gains require either (a) a dedicated Phase B Option C sprint with 60-minute soak test, (b) quality compromises (Q3 quantisation, top- $k=4$, expert pruning), or (c) hardware upgrade — all outside the scope of this work.

1 Background

The original technical report [?] established a 4-node Pi5 cluster running Qwen3-30B-A3B Q40 at 12.71 tok/s sustained, with 8 source-level patches to `distributed-llama`. A follow-up session (Stage 9) applied TIER 0 sysctls — GRO disable, raised TCP/socket buffers, swappiness $\rightarrow 1$ — and pushed the cluster to 14.011 tok/s. The A2 op-fusion (OP_SILU_MUL combining the MoE FFN’s silu+mul ops with a single barrier per expert per layer) added another 0.5% bit-exact gain to 14.081 tok/s. After these gains, the question became: *can we push further under strict bit-exact, no-kernel-rebuild, no-quality-loss constraints?*

This companion note documents the Rounds 3 and 4 investigation and its empirical conclusion: **the cluster is at its bit-exact ceiling under the operational constraints.**

2 Methodology

2.1 Constraints

- Strict bit-exact (verified per-token output for a deterministic prompt at `temperature=0`).
- No kernel rebuild (production Pi OS 6.12.75).
- No reboot of the cluster (`cmdline.txt` unchanged).
- Hardware unchanged (4× Pi 5 16 GB, Gigabit Ethernet through PoE switch).
- No model quality compromise: top-*k* unchanged (8), no quantisation change, no expert pruning, no speculative decoding.

2.2 Investigation strategy

Round 3 launched four parallel background research agents:

1. **Linux Plumbers / SOSP / OSDI / EuroSys / USENIX ATC papers 2024–2026.**
2. **eBPF / XDP / AF_XDP / io_uring / kernel-bypass networking.**
3. **Linux scheduler (CFS, EEVDF in 6.6+) tunings for compute-bound workloads.**
4. **ARM-specific Linux kernel cache, prefetcher, NUMA, and memory bandwidth.**

Each agent returned ranked actionable findings. Round 4 then implemented and benchmarked the top candidates via $n = 20$ paired runs (deterministic prompt, `temperature=0`, 250-token generation, 95% CI computed).

2.3 Benchmark protocol

```
warmup: 2 discarded runs
runs:   n=20 paired
prompt: "Write 200 words about distributed computing"
params: max_tokens=250, temperature=0
metric: tokens / wall-clock seconds
```

3 Results

3.1 Benchmark trajectory through the investigation

Table 1: Measured throughput across the optimisation journey.

Stage	Mean (tok/s)	±95% CI	Cumulative
b4rtaz #255 public ceiling (Pi5 8GB)	13.04	—	baseline
Stage 1–8 baseline ([?])	13.720	0.050	+5.2%
Stage 9 — TIER 0 sysctls (GRO off, buffers, swap)	14.011	0.051	+7.45%
A2 — OP_SILU_MUL fusion (bit-exact)	14.081	0.055	+7.99%
Round 3a — additional sysctls (autogroup, autocorking, etc)	14.108	0.056	noise
Round 3b — madvise weights (RANDOM/DONTFORK)	14.027	0.044	noise
Round 3c — per-thread CPU affinity 1-to-1	14.023	0.035	noise
Round 3d — threaded NAPI + deferred IRQ	12.904	—	regression −3.9%
Round 4a — EEVDF sched_setattr slice 4ms	14.061	0.048	noise
Round 4b — vm.dirty_bytes cap 16MB	14.027	0.044	noise
Round 4c — TCP buffers 8MB → 32MB	13.968	0.038	noise
Round 4d — Phase B foundation + -tile-sync 4	12.904	—	regression −8%
Round 4e — max-seq-len 32768 → 4096	13.965	0.027	noise
Final (cold cluster, all bit-exact changes applied)	14.046	0.049	+2.38% vs baseline +7.72% vs #255

3.2 Bottleneck characterisation via ARM PMU

Profiling with `perf stat` on both root and worker nodes during sustained inference:

Table 2: Measured stalls and DRAM traffic (per node, 60s sample, n=4 nodes).

Metric	Root (rpi-1005)	Worker (rpi-1006)
stalled-cycles-backend (% cycles)	49.25%	47.69%
stalled-cycles-frontend (% cycles)	3.83%	—
DRAM read bandwidth ($l2d_cache_refill \times 64$)	2.74 GB/s	2.61 GB/s
DRAM write bandwidth ($l2d_cache_wb \times 64$)	8.66 GB/s	8.47 GB/s
Total DRAM bandwidth per node	11.40 GB/s	11.08 GB/s
IPC (insn / cycle)	1.74	—
dTLB miss rate	0.08%	—
iTLB miss rate	0.01%	—
branch-misses	0.07%	—

The 49% backend stall, combined with ~ 11 GB/s per-node DRAM bandwidth ($\approx 85\%$ of the practical 13 GB/s 4-thread Triad ceiling for LPDDR4X-4267), is the diagnostic smoking gun: the cluster spends roughly half its cycles waiting on memory. Compute-side optimisations (NEON ILP, register tile fusion, thread placement) yield zero measurable gain because CPUs are already idle waiting on DRAM half the time.

The $3.16\times$ write-to-read ratio (8.66 GB/s writes vs 2.74 GB/s reads) is unusual for streaming-weight inference and points to the multi-pass intermediate buffer pattern in the MoE FFN ($w_3 \rightarrow \text{silu} \rightarrow \text{mul}$ produces and writes back temporary buffers between barriers). The A2 OP_SILU_MUL fusion already shipped reduces this by 224 barriers/token, but the underlying memory traffic remains.

4 Round 3 and Round 4 findings

4.1 Findings from research agents

Table 3: Top findings from 4 background research agents, with applicability to our setup.

Technique	Source	Applicable?	Status
IRQ suspension (<code>irq_suspend_timeout</code>)	LWN 1008399	No (kernel 6.13+)	Pi5 on 6.12
MADV_HUGEPAGE	arxiv 2407.13476 et al.	No (THP not compiled in Pi OS)	Tested no-op
AF_XDP zero-copy	kernel docs	No (macb no native XDP)	RFC only
io_uring SQPOLL + DEFER_TASKRUN	axboe/liburing	Net negative on 4-core	Skipped
S0_BUSY_POLL + RPS/RFS	LWN 540281	Already TIER 0	Applied
Threaded NAPI	5.12+ docs	Tested -3.9%	Reverted
SCHED_FIFO / RR at moderate priority	kernel.org	Considered, deferred	Risk:reward poor
NUMA fake=4 + local	Linux NUMA docs	Requires reboot	Excluded
MPAM cache partitioning	docs.kernel.org	Kernel 6.19+ AND A76 lacks hw	Future kernel
Per-task EEVDF <code>sched_runtime</code>	LWN 969062	Applied (4ms)	No measurable gain
ik_llama pair-row matmul ILP	PR #229 follow-up	Tested no-op (OoO already optimal)	Reverted
ik_llama PR #1485 cross-expert pool	ik_llama	Already-optimal in dllama	N/A
Jumbo frames MTU 9000	RFC 2861 + macb	bcmgenet EBUSY live link	Documented dead-end
Phase B foundation cherry-pick (K=0)	opt-b5b6-tiled-sync	Applied (perf-neutral)	Foundation in repo
Phase B -tile-sync 4 (no Option C wiring)	opt-b5b6-tiled-sync	Tested -8%	Reverted

4.2 Phase B foundation cherry-pick

We cherry-picked four commits from the archived `opt-b5b6-tiled-sync` branch onto `pi5-cluster`:

- `ce7b96e feat(B5+B6): tile-sliced sync behind -tile-sync flag`
- `6ebd00f fix: writeReadMany → writeMany+readMany`
- `469101e feat(B5+B6): async sync orchestrator (drain thread)`
- `2aba914 feat(B6): matmul_Q80_Q40_F32_tiled wrapper`

With `-tile-sync 0` (default) the cluster behaviour is byte-identical to the pre-foundation state. We measured 14.092 ± 0.044 tok/s with `K=0`, confirming perf-neutrality. With `-tile-sync 4` active, we measured -8% regression: the wrapper subdivides syncs into `K` small writes but *does not yet drive the matmul to emit per-tile signals* (that is the Option C wiring, ~40 LOC still pending). The TCP small-write penalty without overlap dominates.

4.3 EEVDF per-task slice tuning

We added a `sched_setattr()` call at the start of `executorWorkerLoop` requesting a 4ms slice (default 700us under EEVDF in 6.12). We confirmed via `/proc/<pid>/sched` that workers reported `se.slice = 4000000`. Measurement: 14.061 ± 0.048 tok/s, no measurable gain vs the unset case. Hypothesis: with 4 pinned worker threads, performance governor, and minimal system load, the default scheduling cadence was already near-optimal.

4.4 Sysctls applied (cumulative, persisted)

`/etc/sysctl.d/99-dllama-r3.conf` (Round 3):

```
kernel.sched_autogroup_enabled = 0
net.ipv4.tcp_autocorking = 0
net.ipv4.tcp_slow_start_after_idle = 0
net.ipv4.tcp_no_metrics_save = 1
```

/etc/sysctl.d/99-dllama-r4.conf (Round 4):

```
net.ipv4.tcp_thin_linear_timeouts = 1
net.core.netdev_max_backlog = 8192
net.core.rmem_max = 33554432
net.core.wmem_max = 33554432
net.ipv4.tcp_rmem = 4096 87380 33554432
net.ipv4.tcp_wmem = 4096 65536 33554432
vm.dirty_bytes = 16777216
vm.dirty_background_bytes = 4194304
```

Each individual change produced no measurable gain (within ± 0.15 tok/s variance floor), but they are persisted as defensive baseline tunings consistent with Linux Plumbers 2024/2025 networking microconf guidance.

5 Discussion

5.1 Why no further bit-exact improvement?

The Pi5 LPDDR4X-4267 memory controller has a theoretical peak of 17.07 GB/s and a measured 4-thread Triad ceiling of ~ 13 GB/s. Our four nodes each sustain ~ 11 GB/s during inference, or $\approx 85\%$ of practical peak. The compute path ($Q80 \times Q40$ matmul with NEON dotprod) is hardware-tuned to the limit; the synchronisation path (TCP all-to-all-broadcast over 1 GbE) is latency-bound at ~ 125 us per sync round. Together they saturate the system’s effective work rate.

Conclusion: with the constraints of bit-exact computation, no kernel rebuild, and no quality compromise, the practical ceiling for this hardware running Qwen3-30B-A3B Q40 is ~ 14.0 tok/s. We achieve 14.046 tok/s reproducibly when the cluster is cold-started.

5.2 What would unlock further gains?

1. **Phase B Option C (~ 40 LOC + 60 min soak):** wire the existing async-sync drain thread to the matmul tile-signal emitter so that the network ships tile t while the matmul computes tile $t + 1$. Estimated +5–12% based on the measured 17% sync wall and 60% overlap target. *Foundation is now in pi5-cluster*; only wiring + watchdog remains.
2. **Quality compromise:** top- $k=4$ (arxiv:2604.07035) gives +25–40% on a memory-bandwidth-bound MoE workload, but at the cost of ~ 1 –2 percentage points on reasoning benchmarks. Q3_K_S in dllama (would need NEON kernel port) projects +25–35%. REAP 25% expert pruning (offline, no LOC change) projects +15–25%, with -1.1% on math benchmarks.
3. **Hardware:** Apple Silicon UMA (M-series) or any platform with >30 GB/s effective per-core DRAM. Out of scope.

5.3 Implications for Pi5 LLM workloads more broadly

The Pi5 has become a popular substrate for “edge LLM” experiments. Our findings suggest:

- For bandwidth-bound workloads (any model larger than ~ 1 GB of active weights per token per node), the LPDDR4X memory subsystem is the binding constraint. NEON kernels, compiler flags, and scheduler tuning have already saturated.
- MoE architectures help disproportionately on Pi5: the same total parameter count with sparse activation maps better to the memory budget than dense.
- Network-side tuning (TCP buffers, busy-poll, IRQ affinity) is largely solved in TIER 0; further work on this axis returns within the measurement noise.

- Compute/comms overlap (Phase B Option C, FlashOverlap, CommFuse) is the only software-side lever that can hide the memory latency, and it requires dedicated engineering with a safety net (watchdog + soak test).

6 Reproducibility

All commits are pushed to [danielcorrea-hellomatik/distributed-llama](https://github.com/danielcorrea-hellomatik/distributed-llama) on the `pi5-cluster` branch:

SHA	Change
162cd83	docs: SESSION Round 4 final
2aba914	feat(B6): matmul_Q80_Q40_F32_tiled wrapper
469101e	feat(B5+B6): async sync orchestrator
6ebd00f	fix: writeReadMany → writeMany+readMany
ce7b96e	feat(B5+B6): <code>-tile-sync</code> K flag
850d1e1	perf(executor): EEVDF <code>sched_setattr</code> slice 4ms
f5bd00c	docs: SESSION Round 3 (4 background agents)
33ef826	docs: SESSION + pair-row matmul ROI 0%
c0bcf74	docs: SESSION EXTENDED initial
106eab3	perf(moe): OP_SILU_MUL bit-exact (+0.5%)
d50d30c	docs(stage9): TIER 0 sysctls (+1.5%)

Cluster operational state and full session logs live in `docs/SESSION-2026-05-17-EXTENDED.md` (488 lines). Per-Pi persisted tunings in `/etc/sysctl.d/99-dllama.conf` (TIER 0), `99-dllama-r3.conf` (Round 3), `99-dllama-r4.conf` (Round 4), and `/etc/systemd/system/dllama-eth-tune.service` (`ethtool -K eth0 gro off` at boot).

7 Conclusion

We documented a multi-round empirical investigation pushing a 4-node Raspberry Pi 5 cluster running Qwen3-30B-A3B Q40 from its public ceiling of 13.04 tok/s up to 14.046 tok/s sustained, a **+7.72% improvement** over the state-of-the-art for this exact hardware. The journey involved:

- 8 source-level patches to `distributed-llama` (Stages 1–8).
- TIER 0 sysctl bundle (Stage 9), validated at +2.12% over the patched baseline.
- OP_SILU_MUL op-fusion (A2), bit-exact +0.5%.
- Phase B async-sync foundation cherry-picked but not activated; Option C wiring remains as future work.
- EEVDF per-task slice tuning, two rounds of sysctl bundles, IRQ affinity, per-thread pinning, and madvise hints — none of which provided measurable additional gain.
- Empirical confirmation via ARM PMU that the cluster is DRAM-bandwidth-bound (49% backend stalls).

The cluster is now at its practical bit-exact ceiling under the operational constraints. We document the Phase B Option C wiring path as the next sprint candidate (~40 LOC + watchdog + 60-minute soak test) with an estimated upside of +5–12%. Beyond that, further gains require quality compromise or hardware upgrade.

References

- [1] D. Correa Villa, “Distributed LLM Inference on a 4-Node Raspberry Pi 5 Cluster,” Hellomatik technical report, 16 May 2026. (`paper/main_en.tex` in repo)
- [2] B. Tadych, *distributed-llama*, <https://github.com/b4rtaz/distributed-llama>.
- [3] *Qwen3-30B-A3B on 4×Pi5 8GB — 13.04 tok/s discussion*, <https://github.com/b4rtaz/distributed-llama/discussions/255>.

- [4] J. Damato, “Smarter IRQ suspension,” LWN 1008399, 2025. <https://lwn.net/Articles/1008399/>
- [5] “EEVDF and the per-task slice control,” LWN 969062. <https://lwn.net/Articles/969062/>
- [6] Linux Plumbers Conference 2025 networking microconf. <https://lpc.events/event/19/sessions/236/>
- [7] I. Kawrakow, *ik_llama.cpp*, https://github.com/ikawrakow/ik_llama.cpp.
- [8] D. Correa Villa, “Phase B plan and wiring,” `docs/PHASE-B-PLAN.md` & `docs/PHASE-B-WIRING.md` in repo, 2026.
- [9] K. Hong, X. Li *et al.*, “FlashOverlap,” *arXiv:2504.19519*, 2025.
- [10] “CommFuse: Hiding tail latency via communication decomposition,” *arXiv:2604.24013*, 2026.
- [11] “Gemma 4, Phi-4, Qwen3 accuracy-efficiency tradeoffs,” *arXiv:2604.07035*, 2026.
- [12] Linux kernel NAPI documentation. <https://docs.kernel.org/networking/napi.html>